PrepPressure — Team Coordination & Ownership (v3)

> Commit as `docs/TEAM-OWNERSHIP.md`.

1. Who owns what

Member	Area	Side
Badr Al Jerf	Backend platform (`main.ts`, `app.module.ts`, config, Swagger `/docs`), auth (register/login/JWT/global guard), users, profile statistics	Backend
Esmaeel Mahfoud	Attempts: start, timing, answers, finish, scoring, results, my-attempts list	Backend
Shiam Ezzo	Questions module, interviews module, `docs/TEST-PLAN.md`, final full-flow e2e test	Backend + QA
Laith Haj Hosin	Frontend shell (entry, router, layout, auth context), login/register, take-interview, results, profile pages; contract coordinator	Frontend
Adham Albasha	Frontend questions pages and interviews pages	Frontend
Hassan Alloush	No tasks assigned while unavailable	—

Every person writes the tests for their own code. Shiam additionally owns the test plan and the final end-to-end test.

2. The “nobody waits” rules

1. **Contract first, once.** `docs/API-CONTRACT.md` is committed on day 1, exactly as delivered. After that, nobody waits for anybody’s code: you only need the contract.
2. **One owner per file.** The full file list is in each person’s task file. If a file isn’t in your list, you don’t edit it.
3. **No cross-module imports in the backend.**
   - Authentication is a **global guard** (Badr). Other modules don’t import it; they just read `request.user.userId`.
   - When a module needs another module’s data, it reads that collection directly with `@InjectConnection()` + `connection.collection('<name>')`, using only fields from contract section 8.
   - Each module has its own small numeric-id helper using its own key in `counters`.
   - Each controller adds `@ApiTags(...)` and `@ApiBearerAuth()` itself (from `@nestjs/swagger`, a library — not a teammate’s file).
4. **No cross-feature imports in the frontend.**
   - Each feature folder has its own `api.ts`, `types.ts` (copied from the contract) and `mocks.ts`.
   - Small duplication (a 15-line fetch helper per feature) is accepted on purpose so nobody depends on anybody.
   - Features talk to each other only through **URLs** (e.g. Adham’s “Start” button is a link to `/interviews/:id/take`) and the **localStorage keys / logout event** from contract section 9.
5. **Test alone.**
   - Backend: each e2e test builds a Nest testing module with **only your module**, an in-memory or test MongoDB, and a **fake global guard** that sets `request.user = { userId: 1, username: 'tester' }` (switch the id to test ownership). When you need data from another collection, insert raw documents with the contract shape in your test setup.
   - Frontend: run with `VITE_USE_MOCKS=true`.

3. The only shared-file exception: registration lines

Two files must list modules/pages from other people:

File	Owner	What others add
`backend/src/app.module.ts`	Badr	Esmaeel adds `AttemptsModule`; Shiam adds `QuestionsModule` and `InterviewsModule` — **one import line + one entry in `imports: []` each**, in their own PR
`frontend/src/app/router.tsx`	Laith	Adham adds his route entries — **one import line per page + one route object per page**, in his own PR

Nothing else in these files may be changed by non-owners. These lines are added in the PR that brings the module/page, so the build never references a file that doesn't exist yet. Because the edits are tiny and in different spots, merge conflicts are trivial.

4. Repository layout

```
preppressure/
  backend/
    src/
      main.ts, app.module.ts, config/                    (Badr)
      auth/, users/                                      (Badr)
      questions/, interviews/                            (Shiam)
      attempts/                                          (Esmaeel)
    test/
      auth.e2e-spec.ts, users.e2e-spec.ts                (Badr)
      questions.e2e-spec.ts, interviews.e2e-spec.ts,
      full-flow.e2e-spec.ts                              (Shiam)
      attempts.e2e-spec.ts, scoring.spec.ts              (Esmaeel)
    .env.example                                         (Badr)
  frontend/
    src/
      main.tsx, App.tsx, app/, layout/, features/auth/,
      features/attempts/, features/profile/              (Laith)
      features/questions/, features/interviews/          (Adham)
    .env.example                                         (Laith)
  docs/
    API-CONTRACT.md, TEAM-OWNERSHIP.md, SPECIFICATION.md  (frozen; changes via team PR, Laith coordinates)
    TEST-PLAN.md                                         (Shiam)
  README.md                                              (see section 6)
  .gitignore                                             (Badr)
```

If the existing repository already has some of these files with different names, keep the existing name and update this table.

5. Git workflow

- Branches: `feature/<area>-<short-name>`, e.g. `feature/attempts-scoring`. Small PRs, no direct pushes to `main`.
- PR description: what was done, tests run, contract impact (should be "none").
- Never commit `.env`, `node_modules`, `dist`, database dumps, real secrets. Commit `.env.example` only.
- Review: any teammate can review; contract changes need everyone.

6. README.md (shared document, split by section)

To avoid edits on the same lines, `README.md` has fixed sections, each with one owner:

Section heading	Owner
`## Overview` and `## Project structure`	Laith
`## Backend setup`, `## Environment variables`, `## Swagger (/docs)`	Badr
`## Questions and interviews` (incl. index notes)	Shiam
`## Attempts, timing and scoring`	Esmaeel
`## Frontend setup`	Laith

`## Frontend pages`	Adham
`## Testing`	Shiam

Laith creates the README with all empty headings on day 1 (5 minutes); then each person writes only under their heading.

7. Timeline (parallel)

Day	Everyone in parallel
1	Contract + README skeleton committed. Each person starts their own files immediately.
2–5	Build and test alone (fake guard / mocks). Open PRs as parts finish.
6	Integration day: merge all, set `VITE_USE_MOCKS=false`, run Shiam's full-flow test, fix mismatches against the contract.
7	Demo rehearsal (in English): each member explains the architecture and their part.

8. Demo checklist

Register → login → Authorize in Swagger `/docs` → create questions of all 3 types → create an interview with 1 question and one with several → second user sees them but cannot edit (403) → second user takes an interview → half credit shown → timeout case → result page → profile statistics → edit a question and show the old result unchanged → restart backend, data still there.